

Join Method and IsAlive Property of Thread Class in C# with Examples

In this article, I am going to discuss the use of the **Join Method and IsAlive Property of Thread Class in C#** with Examples. Please read our previous article where we discussed [How to Return Data from a Thread Function in C# using the Callback method](#) with an Example. As part of this article, we are going to discuss the following pointers.

1. **Understanding the need for Join Method of Thread Class in C#.**
2. **Examples using different overloaded versions of the Join Method.**
3. **Examples to Understand the use of IsAlive Property of Thread Class in C#.**

Join Method of Thread Class in C#.

Let us understand the use of the Join Method of Thread Class in C# with Examples. For a better understanding, please have a look at the following example. In the below example we have created three methods and then execute these three methods using three different threads. The point that you need to remember is the threads thread1, thread2, and thread3 are called the child threads of the main thread. This is because these three threads are created by the main thread only. Here, thread1 executes Method1, thread2 executes Method2, and thread3 executes Method3. Further, if you notice, we have delayed the execution of Method1 by 3 seconds, Method2 execution by 2 seconds, and Method3 execution by 5 seconds using the Thread class static Sleep method which takes the time in milliseconds and will make the current thread sleep for that millisecond. Here, we are using the overloaded version of the Thread class constructor which takes the ThreadStart delegate as a parameter.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            Thread.Sleep(3000);
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
    }
}
```

```

    }
    static void Method2()
    {
        Console.WriteLine("Method2 - Thread2 Started");
        Thread.Sleep(2000);
        Console.WriteLine("Method2 - Thread2 Ended");
    }
    static void Method3()
    {
        Console.WriteLine("Method3 - Thread3 Started");
        Thread.Sleep(5000);
        Console.WriteLine("Method3 - Thread3 Ended");
    }
}
}

```

Now, run the application and see the output. The output may vary on your machine when you run the application.

```

Main Thread Started
Main Thread Ended
Method1 - Thread1 Started
Method3 - Thread3 Started
Method2 - Thread2 Started
Method2 - Thread2 Ended
Method1 - Thread 1 Ended
Method3 - Thread3 Ended

```

As you can see from the above output, the Main thread is not waiting for all the child threads to complete their execution or task. If you want the Main thread should not be exited until and unless all the child thread or any of the child threads completes their task then you need to use the Join method of the Thread class in C#.

Join Method of Thread Class in C#:

The Join Method of the Thread Class in C# blocks the current thread and makes it wait until the child thread on which the Join method invoked completes its execution. There are three overloaded versions available for the Join Method in Thread class as shown below.

```

public void Join();

public bool Join(int millisecondsTimeout);

public bool Join(TimeSpan timeout);

```

The following are the definitions of the above three overloaded versions of the Join Method of Thread class in C# provided by Microsoft:

1. **Join():** This method blocks the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping. It will throw ThreadStateException if the caller attempted to join a thread that is in the System.Threading.ThreadState.Unstarted state.
2. **Join(int millisecondsTimeout):** This method blocks the calling thread until the thread represented by this instance terminates or the specified time elapses while continuing to perform standard COM and SendMessage pumping. The parameter millisecondsTimeout specifies the number of milliseconds to wait for the thread to terminate. It returns true if the thread has terminated; false if the thread has not terminated after the amount of time specified by the millisecondsTimeout parameter has elapsed. It will throw ArgumentOutOfRangeException if the value of millisecondsTimeout is negative and is not equal to System.Threading.Timeout.Infinite in milliseconds. It will throw ThreadStateException if the thread has not been started.
3. **Join(TimeSpan timeout):** This method blocks the calling thread until the thread represented by this instance terminates or the specified time elapses while continuing to perform standard COM and SendMessage pumping. Here, the parameter timeout specifies a System.TimeSpan is set to the amount of time to wait for the thread to terminate. It returns true if the thread is terminated; false if the thread has not terminated after the amount of time specified by the timeout parameter has elapsed. It throws ArgumentOutOfRangeException if the value of timeout is negative and is not equal to System.Threading.Timeout.Infinite in milliseconds, or greater than System.Int32.MaxValue milliseconds. It throws ThreadStateException if the caller attempted to join a thread that is in the System.Threading.ThreadState.Unstarted state.

We can simplify the above definitions as follows:

Join(): The first version of the Join method which does not take any parameter will block the calling thread (i.e. the Parent thread) until the thread (child thread) completes its execution. In this case, the calling thread is going to wait for an indefinite time until the thread on which the Join Method is invoked is completed.

Join(int millisecondsTimeout): The second version of the Join Method allows us to specify the time out. It means it will block the calling thread until the child thread terminates or the specified time elapses. This overloaded takes time in milliseconds. This method returns true if the thread has terminated and returns false if the thread has not terminated after the amount of time specified by the millisecondsTimeout parameter has elapsed.

Join(TimeSpan timeout): The third overloaded version of this method is the same as the second overloaded version. The only difference is that here we need to use TimeSpan to set the amount of time to wait for the thread to terminate.

Example to Understand the Join Method of Thread Class in C#

For a better understanding of how to use the Thread Class Join Method in C#, please have a look at the below example. In the below example, we have called the Join method on all three threads which means it will block the main thread until all the child threads complete their tasks.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        Console.WriteLine("Main Thread Started");
        //Main Thread creating three child threads
        Thread thread1 = new Thread(Method1);
        Thread thread2 = new Thread(Method2);
        Thread thread3 = new Thread(Method3);
        thread1.Start();
        thread2.Start();
        thread3.Start();
        thread1.Join(); //Block Main Thread until thread1 completes its execution
        thread2.Join(); //Block Main Thread until thread2 completes its execution
        thread3.Join(); //Block Main Thread until thread3 completes its execution
        Console.WriteLine("Main Thread Ended");
        Console.Read();
    }
    static void Method1()
    {
        Console.WriteLine("Method1 - Thread1 Started");
        Thread.Sleep(1000);
        Console.WriteLine("Method1 - Thread 1 Ended");
    }
    static void Method2()
    {
        Console.WriteLine("Method2 - Thread2 Started");
        Thread.Sleep(2000);
        Console.WriteLine("Method2 - Thread2 Ended");
    }
    static void Method3()
    {
        Console.WriteLine("Method3 - Thread3 Started");
        Thread.Sleep(5000);
        Console.WriteLine("Method3 - Thread3 Ended");
    }
}
}

```

Now, run the above code and you will see that the Main thread is started first and will wait until all the child threads complete their execution as shown in the below output.

```

Main Thread Started
Method1 - Thread1 Started
Method3 - Thread3 Started
Method2 - Thread2 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Method3 - Thread3 Ended
Main Thread Ended

```

Now, for example, if you don't want the main thread to wait until thread3 completes its execution. Then you just need to call the Join method on thread1 and thread2 as shown in the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();
            thread1.Join(); //Block Main Thread until thread1 completes its execution
            thread2.Join(); //Block Main Thread until thread2 completes its execution
            //Now, Main Thread will not wait for thread3 to complete its execution
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            Thread.Sleep(1000);
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
        static void Method2()
        {
            Console.WriteLine("Method2 - Thread2 Started");
            Thread.Sleep(2000);
            Console.WriteLine("Method2 - Thread2 Ended");
        }
        static void Method3()
        {
            Console.WriteLine("Method3 - Thread3 Started");
            Thread.Sleep(5000);
            Console.WriteLine("Method3 - Thread3 Ended");
        }
    }
}

```

Output:

```
Main Thread Started
Method3 - Thread3 Started
Method1 - Thread1 Started
Method2 - Thread2 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Main Thread Ended
Method3 - Thread3 Ended
```

Other Overloaded Versions of Thread Class Join Method in C#:

You need to use the second and third overloaded version of the Thread Class Join Method in C# when you want the main thread to wait for a specified amount of time. For example, you want the main thread to wait for 3 seconds for thread2 and thread3 to complete their task. Then you need to use the Join method as shown below in the below example. Remember the overloaded versions which take time in milliseconds and TimeSpan returns a boolean value indicating whether the thread completes its execution or not. Boolean true means method execution completed and false means method execution not completed.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Main Thread creating three child threads
            Thread thread1 = new Thread(Method1);
            Thread thread2 = new Thread(Method2);
            Thread thread3 = new Thread(Method3);
            thread1.Start();
            thread2.Start();
            thread3.Start();

            //Now, Main Thread will block for 3 seconds and wait thread2 to co
execution
            if (thread2.Join(TimeSpan.FromSeconds(3)))
            {
                Console.WriteLine("Thread 2 Execution Completed in 3 second");
            }
            else
            {
                Console.WriteLine("Thread 2 Execution Not Completed in 3 secon
            }
            //Now, Main Thread will block for 3 seconds and wait thread3 to co
execution
            if (thread3.Join(3000))
            {
```

```

        Console.WriteLine("Thread 3 Execution Completed in 3 second");
    }
    else
    {
        Console.WriteLine("Thread 3 Execution Not Completed in 3 second");
    }
    Console.WriteLine("Main Thread Ended");
    Console.Read();
}
static void Method1()
{
    Console.WriteLine("Method1 - Thread1 Started");
    Thread.Sleep(1000);
    Console.WriteLine("Method1 - Thread 1 Ended");
}
static void Method2()
{
    Console.WriteLine("Method2 - Thread2 Started");
    Thread.Sleep(2000);
    Console.WriteLine("Method2 - Thread2 Ended");
}
static void Method3()
{
    Console.WriteLine("Method3 - Thread3 Started");
    Thread.Sleep(5000);
    Console.WriteLine("Method3 - Thread3 Ended");
}
}
}

```

Now, run the application and observe the output. You will see that Method2 execution is completed in 3 seconds while Method3 execution is not completed. Further notice after the Main method execution, Method3 is going to complete its execution. So, the Main thread is blocked for 3 seconds and after 3 seconds the Main thread continues its execution.

```

Main Thread Started
Method2 - Thread2 Started
Method1 - Thread1 Started
Method3 - Thread3 Started
Method1 - Thread 1 Ended
Method2 - Thread2 Ended
Thread 2 Execution Completed in 3 second
Thread 3 Execution Not Completed in 3 second
Main Thread Ended
Method3 - Thread3 Ended

```

IsAlive Property of Thread Class in C#:

The IsAlive property gets a value indicating the execution status of the current thread. It returns true if the thread has been started and has not terminated normally or aborted; otherwise, false. That means the IsAlive

property of the Thread class returns true if the thread is still executing else returns false. Let us understand this with an example.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            Thread thread1 = new Thread(Method1);
            thread1.Start();
            if (thread1.IsAlive)
            {
                Console.WriteLine("Thread1 Method1 is still Executing");
            }
            else
            {
                Console.WriteLine("Thread1 Method1 Completed its work");
            }
            //Wait Till thread1 to complete its execution
            thread1.Join();
            if (thread1.IsAlive)
            {
                Console.WriteLine("Thread1 Method1 is still Executing");
            }
            else
            {
                Console.WriteLine("Thread1 Method1 Completed its work");
            }
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 - Thread1 Started");
            //Making thread to sleep for 2 seconds
            Thread.Sleep(TimeSpan.FromSeconds(2));
            Console.WriteLine("Method1 - Thread 1 Ended");
        }
    }
}
```

Output:


```
Main Thread Started  
Thread1 Method1 is still Executing  
Method1 - Thread1 Started  
Method1 - Thread 1 Ended  
Thread1 Method1 Completed its work  
Main Thread Ended
```

In the next article, I am going to discuss [Thread Synchronization in C#](#) with Examples. Here, in this article, I try to explain **how to use the IsAlive Property and Join Method of Thread class in C#** using Examples. I hope you enjoy this IsAlive Property and Join Method of Thread class in C# using the Examples article.